UNIT-4

Planning

4.1 Planning

Planning is a problem solving with the agent using belief about its action and consequences of action to get to a solution by searching through a abstract space of class.

- **Factored representation:** A state is expressed as a set of variables (fluents), not as atomic states.
- **PDDL (Planning Domain Definition Language):** Used to define planning problems with one action schema instead of many ground actions.
- **States:** Conjunctions of ground, functionless atoms (e.g., At(Truck1, Melbourne) ∧ At(Truck2, Sydney)), with:
- Closed-world assumption → anything not listed is false.
- Unique names assumption → different names = different objects.
- Restrictions: no variables (At(x,y)), no negations (¬Poor), no functions (At(Father(Fred), Sydney))
- **Set semantics:** States can be treated as either logical conjunctions or sets of fluents.
- **Actions:** Defined by action schemas, which specify only what changes (solving the frame problem). Everything else is assumed unchanged.
- **Lifted representation:** Action schemas generalize over variables (restricted first-order logic) instead of enumerating all ground actions.
- Action schemas define what actions are possible (ACTIONS(s)) and their outcomes (RESULT(s, a)).
- They solve the frame problem by listing only what changes; everything else is assumed unchanged. Instead of many ground actions, one lifted schema (using variables) represents them all.

Classical planning concentrates on problems where most actions leave most things unchanged.

A set of ground (variable-free) actions can be represented by a single action schema. The schema is a lifted representation—it lifts the level of reasoning from propositional logic to a restricted subset of first-order logic. For example, here is an action schema for flying a plane from one location to another:

Action(Fly(p, from, to),

PRECOND: At(p, from) ∧ Plane(p) ∧ Airport (from) ∧ Airport (to)

EFFECT:¬At(p, from) ∧ At(p, to))

Meaning: A plane p at airport from can fly to airport to

Preconditions: plane must be at from, and both places must be airports.

Effects: remove At(p, from) and add At(p, to).

The schema consists of the action name, a list of all the variables used in the schema, a precondition and an effect. Although we haven't said yet how the action schema converts into logical sentences, think of the variables as being universally quantified.

For example, here is one ground action that results from substituting values for all the variables:

Action(Fly(P_1, SFO, JFK),

PRECOND:At(P_1, SFO) ∧ Plane(P_1) ∧ Airport (SFO) ∧ Airport (JFK)

EFFECT:¬At(P_1, SFO) ∧ At(P_1, JFK))

Action name: Fly(P_1, SFO, JFK) = a specific action where plane P_1 flies from San Francisco (SFO) to New York (JFK).

Precondition (when this action can happen):

- The plane P_1 must currently be at SFO.
- P_1 must indeed be a plane.
- Both SFO and JFK must be airports.
- If any one of these conditions fails, the action cannot be executed.

Effect (what changes after the action):

- ¬At(P_1, SFO) → after flying, plane P_1 is no longer at SFO.
- At(P_1, JFK) → after flying, plane P_1 is now at JFK.
- Everything else in the world stays the same (frame assumption

Precondition: A conjunction of literals (positive or negated) that must hold for an action to be executed.

Effect: A conjunction of literals that defines how the state changes after the action.

Execution rule:

- An action a is available in state s iff s entails the precondition of a.
- In set semantics: all positive literals must be in s, and all negated literals must not be in s.
- **Formal Notation:** (a ∈ ACTIONS(s)) ⇔ s |= PRECOND(a)

  (variables in a are universally quantified).

where any variables in a are universally quantified. For example,

∀ p, from, to (Fly(p, from, to) ∈ ACTIONS(s)) ⇔

s |= (At(p, from) ∧ Plane(p) ∧ Airport (from) ∧ Airport (to))

An action is applicable in a state if its preconditions are satisfied by that state.

- **Example:** If the action is Fly(x, from, to) and the precondition is At(x, from) ∧ Plane(x), then it's applicable only if the state contains facts matching those conditions.
- When actions are schemas with variables, they can be instantiated in multiple ways:
    - Fly(P_1, SFO, JFK)
    - Fly(P_2, JFK, SFO)
- Both are valid instantiations if their preconditions match the initial state.

Complexity of Finding Applicable Actions

- Suppose an action schema has v variables.
- The domain has k unique object names (e.g., planes, airports).
- Then, in the worst case, we need to check O(vk) instantiations to see which are applicable.-This can explode combinatorially when both v and k are large.

Propositionalization

- Sometimes, planners propositionalize the PDDL problem:
    - Replace each action schema with all its ground instances.
    - Then solve it as a propositional problem, e.g., with SATPLAN.
- Problem: If v and k are large, propositionalization produces too many actions to handle efficiently.
- The result function is defined as:
    - $RESULT(s,a)=(s-DEL(a))\cup ADD(a)$
- Start with current state s.
- Remove fluents (facts) from the delete list (DEL(a)).
- Add fluents from the add list (ADD(a)).

4.1.2 Example: Air cargo transport

$Init(At(C_1, SFO) \land At(C_2, JFK) \land At(P_1, SFO) \land At(P_2, JFK)$
$\land Cargo(C_1) \land Cargo(C_2) \land Plane(P_1) \land Plane(P_2)$
$\land Airport(JFK) \land Airport(SFO))$
$Goal(At(C_1, JFK) \land At(C_2, SFO))$
$Action(Load(c, p, a),$
PRECOND: $At(c, a) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $\neg At(c, a) \land In(c, p))$
$Action(Unload(c, p, a),$
PRECOND: $In(c, p) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $At(c, a) \land \neg In(c, p))$
$Action(Fly(p, from, to),$
PRECOND: $At(p, from) \land Plane(p) \land Airport(from) \land Airport(to)$
EFFECT: $\neg At(p, from) \land At(p, to))$

Figure 10.1 A PDDL description of an air cargo transportation planning problem.

An air cargo transport problem involving loading and unloading cargo and flying it from place to place. The problem can be defined with three actions: Load , Unload, and Fly. The actions affect two predicates: In(c, p) means that cargo c is inside plane p, and At(x, a) means that object x (either plane or cargo) is at airport a. Note that some care must be taken to make sure the At predicates are maintained properly. When a plane flies from one airport to another, all the cargo inside the plane goes with it. In first-order logic it would be easy to quantify over all objects that are inside the plane. But basic PDDL does not have a universal quantifier, so we need a different solution. The approach we use is to say that a piece of cargo ceases to be At anywhere when it is In a plane; the cargo only becomes At the new airport when it is unloaded. So At really means "available for use at a given location."

The following plan is a solution to the problem:

$$[Load(C_1, P_1, SFO), Fly(P_1, SFO, JFK), Unload(C_1, P_1, JFK),$$
$$Load(C_2, P_2, JFK), Fly(P_2, JFK, SFO), Unload(C_2, P_2, SFO)] .$$

Core Actions

The domain uses just three actions:

Load(c, p, a) → put cargo c into plane p at airport a.

Unload(c, p, a) → remove cargo c from plane p at airport a.

Fly(p, from, to) → move plane p from one airport to another.

Key Predicates

Only two predicates drive the planning:

In(c,p): cargo c is inside plane p.

At(x,a): object x (either cargo or plane) is at airport a.

The Subtlety with At

- In the real world, when a plane flies:
    - All cargo inside the plane should "move along with it."
- In first-order logic, that's easy to express with a universal quantifier:
    - ∀ ((,)→ (,))∀c(In(c,p)→At(c,to))
- But basic PDDL has no universal quantifier.

Workaround in PDDL:

- When cargo is loaded, it loses its At(c, a) fact.
- Cargo is not considered At anywhere while it is In a plane.
- Only when it is unloaded does it regain an At(c, a) fact.

So in PDDL:

At means "available for use at a location", not "physically located".

Solution Plan

The given plan achieves the goal:

- Load(C_1, P_1, SFO)
- Fly(P_1, SFO, JFK)
- Unload(C_1, P_1, JFK)
- Load(C_2, P_2, JFK)
- Fly(P_2, JFK, SFO)
- Unload(C_2, P_2, SFO)

At the end:

- C_1 is at JFK.
- C_2 is at SFO.

Goal satisfied.

Spurious Actions

Finally, there is the problem of spurious actions such as Fly(P_1, JFK, JFK), which should be a no-op, but which has contradictory effects (according to the definition, the effect would include At(P_1, JFK) ∧ ¬At(P_1, JFK)). It is common to ignore such problems, because they seldom cause incorrect plans to be produced. The correct approach is to add inequality preconditions saying that the from and to airports must be different;

Example: Fly(P_1, JFK, JFK).

- This should be a no-op (nothing changes).
- But in the add/delete model:

- Add list: At(P_1, JFK)
- Delete list: ¬At(P_1, JFK)
- → Contradiction!

Two ways to handle:

Practical approach: ignore — they rarely break planning.

Correct approach: add inequality preconditions (from ≠ to) to forbid such actions.

4.1.2 Example: The spare tire problem

$$
\begin{array}{l}
Init(Tire(Flat) \land Tire(Spare) \land At(Flat, Axle) \land At(Spare, Trunk)) \\
Goal(At(Spare, Axle)) \\
Action(Remove(obj, loc), \\
\quad \text{PRECOND: } At(obj, loc) \\
\quad \text{EFFECT: } \neg At(obj, loc) \land At(obj, Ground)) \\
Action(PutOn(t, Axle), \\
\quad \text{PRECOND: } Tire(t) \land At(t, Ground) \land \neg At(Flat, Axle) \\
\quad \text{EFFECT: } \neg At(t, Ground) \land At(t, Axle)) \\
Action(LeaveOvernight, \\
\quad \text{PRECOND:} \\
\quad \text{EFFECT: } \neg At(Spare, Ground) \land \neg At(Spare, Axle) \land \neg At(Spare, Trunk) \\
\qquad \land \neg At(Flat, Ground) \land \neg At(Flat, Axle) \land \neg At(Flat, Trunk))
\end{array}
$$

Figure 10.2 The simple spare tire problem.

The goal is to have a good spare tire properly mounted onto the car's axle, where the initial state has a flat tire on the axle and a good spare tire in the trunk. To keep it simple, our version of the problem is an abstract one, with no sticky lug nuts or other complications. There are just four actions: removing the spare from the trunk, removing the flat tire from the axle, putting the spare on the axle, and leaving the car unattended overnight. We assume that the car is parked in a particularly bad neighborhood, so that the effect of leaving it overnight is that the tires disappear. A solution to the problem is [Remove(Flat,Axle), Remove(Spare,Trunk), PutOn(Spare, Axle)].

- **Initial state:** Flat tire on axle, spare tire in trunk.
- **Goal state:** Good spare tire mounted on axle.
- **Actions:**
  - Remove(Spare, Trunk) – take spare from trunk.
  - Remove(Flat, Axle) – remove flat tire from axle.
  - PutOn(Spare, Axle) – mount spare on axle.
  - LeaveCarOvernight – all tires disappear.
- **Solution plan:** [Remove(Flat, Axle), Remove(Spare, Trunk), PutOn(Spare, Axle)]
- The LeaveCarOvernight action is avoided because it destroys the tires.

4.1.3 Example: The blocks world

$Init(On(A, Table) \land On(B, Table) \land On(C, A)$
$\quad \land Block(A) \land Block(B) \land Block(C) \land Clear(B) \land Clear(C))$
$Goal(On(A, B) \land On(B, C))$
$Action(Move(b, x, y),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Clear(y) \land Block(b) \land Block(y) \land$
$\quad\quad (b \neq x) \land (b \neq y) \land (x \neq y),$
$\quad$ EFFECT: $On(b, y) \land Clear(x) \land \neg On(b, x) \land \neg Clear(y))$
$Action(MoveToTable(b, x),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Block(b) \land (b \neq x),$
$\quad$ EFFECT: $On(b, Table) \land Clear(x) \land \neg On(b, x))$

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.



Figure 10.4 Diagram of the blocks-world problem in Figure 10.3.

One of the most famous planning domains is known as the blocks world. This domain consists of a set of cube-shaped blocks. The blocks can be stacked, but only one block can fit directly on top of another. A robot arm can pick up a block and move it to another position, either on the table or on top of another block. The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it. The goal will always be to build one or more stacks of blocks, specified in terms of what blocks are on top of what other blocks.

We use On(b, x) to indicate that block b is on x, where x is either another block or the table. The action for moving block b from the top of x to the top of y will be Move(b, x, y).Now, one of the preconditions on moving b is that no other block be on it. In first-order logic, this would be ¬∃ x On(x, b) or, alternatively, ∀ x ¬On(x, b). Basic PDDL does not allow quantifiers, so instead we introduce a predicate Clear (x) that is true when nothing is on x.

The action Move moves a block b from x to y if both b and y are clear. After the move is made, b is still clear but y is not. A first attempt at the Move schema is

$$Action(Move(b, x, y),$$
$$\text{PRECOND: } On(b, x) \wedge Clear(b) \wedge Clear(y),$$
$$\text{EFFECT: } On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)) .$$

Unfortunately, this does not maintain Clear properly when x or y is the table. When x is the Table, this action has the effect Clear (Table), but the table should not become clear; and when y =Table, it has the precondition Clear (Table), but the table does not have to be clear for us to move a block onto it. To fix this, we do two things. First, we introduce another action to move a block b from x to the table:

$$Action(MoveToTable(b, x),$$
$$\text{PRECOND: } On(b, x) \wedge Clear(b),$$
$$\text{EFFECT: } On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)) .$$

Second, we take the interpretation of Clear (x) to be "there is a clear space on x to hold a block." Under this interpretation, Clear (Table) will always be true. The only problem is that nothing prevents the planner from using Move(b, x, Table) instead of MoveToTable(b, x). We could live with this problem—it will lead to a larger-than-necessary search space, but will not lead to incorrect answers—or we could introduce the predicate Block and add Block(b)∧ Block(y) to the precondition of Move.

Initial State (Init):

On(A, Table) ∧ On(B, Table) ∧ On(C, A) →

- Block A is on the table.
- Block B is on the table.
- Block C is on top of A.

Block(A) ∧ Block(B) ∧ Block(C) → A, B, and C are blocks.

Clear(C) → Nothing is on C (so C can be moved).

Goal (Goal):

On(A, B) ∧ On(B, C) → Stack the blocks so A is on B and B is on C, forming a tower C → B → A.

Actions:

Move(b, x, y): Move block b from x to y.

Preconditions:

- On(b, x) → b is on x.
- Clear(b) → b has nothing on top.
- Clear(y) → y has nothing on top.

- Block(b) ∧ Block(y) → b and y are blocks.
- b ≠ x, b ≠ y, x ≠ y → cannot move to the same place.

Effects:

- On(b, y) → b is now on y.
- Clear(x) → x becomes clear.
- ¬On(b, x) → b is no longer on x.
- ¬Clear(y) → y is no longer clear.

MoveToTable(b, x): Move block b from x to the table.

Preconditions:

- On(b, x) and Clear(b)
- Block(b)
- b ≠ x

Effects:

- On(b, Table) → b is on the table.
- Clear(x) → x becomes clear.
- ¬On(b, x) → b is no longer on x.

Solution Sequence:

[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]

Step 1: Move C from A to Table → frees A.

Step 2: Move B from Table to C → starts tower.

Step 3: Move A from Table to B → completes tower C → B → A.

4.1.4 The complexity of classical planning

- PlanSAT is the question of whether there exists any plan that solves a planning problem.
- Bounded PlanSAT asks whether there is a solution of length k or less; this can be used to find an optimal plan.
- For classical planning, both are decidable (finite states).
- With function symbols, PlanSAT becomes semidecidable (may not terminate if unsolvable), but Bounded PlanSAT stays decidable.
- Both are in PSPACE (harder than NP).
- Even with restrictions (no negative effects), they are still NP-hard; only when negative preconditions are also disallowed does PlanSAT drop to P.
- In practice, many domains (e.g., blocks world) make PlanSAT easy (P) but Bounded PlanSAT NP-complete—so optimal planning is hard, but finding a plan can be easy.
- Good heuristics are key to solving practical problems efficiently.

4.2 Algorithms for Planning as State-Space Search

- Planning algorithms treat planning as a search problem:
- Forward search: expand from the initial state toward the goal.
- Backward search: start from the goal and work backward to the initial state.
- Declarative action schemas make both directions possible.

4.2.1 Forward (progression) state-space search

First, forward search is prone to exploring irrelevant actions. Consider the noble task of buying a copy of AI: A Modern Approach from an online bookseller.

Suppose there is an action schema Buy(isbn) with effect Own(isbn). ISBNs are 10 digits, so this action schema represents 10 billion ground actions. An uninformed forward-search algorithm would have to start enumerating these 10 billion actions to find one that leads to the goal.

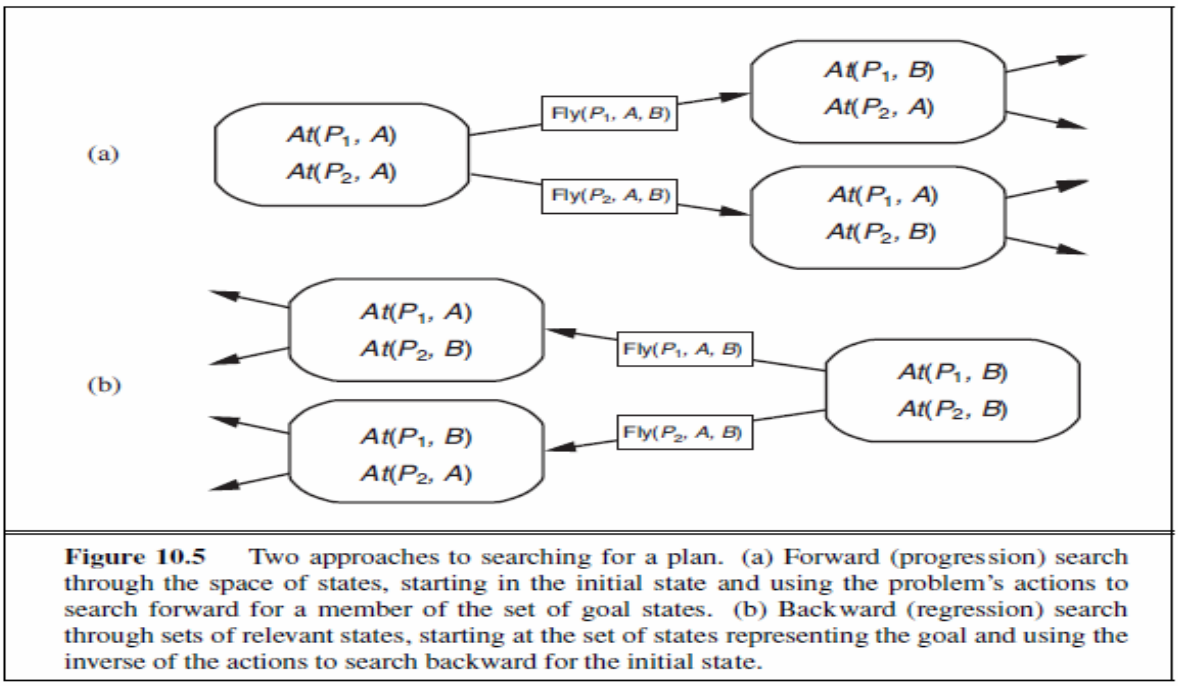


Figure 10.5 Two approaches to searching for a plan. (a) Forward (progression) search through the space of states, starting in the initial state and using the problem's actions to search forward for a member of the set of goal states. (b) Backward (regression) search through sets of relevant states, starting at the set of states representing the goal and using the inverse of the actions to search backward for the initial state.

- Second, planning problems often have large state spaces. Consider an air cargo problem with 10 airports, where each airport has 5 planes and 20 pieces of cargo. The goal is to move all the cargo at airport A to airport B. There is a simple solution to the problem: load the 20 pieces of cargo into one of the planes at A, fly the plane to B, and unload the cargo.
- Finding the solution can be difficult because the average branching factor is huge: each of the 50 planes can fly to 9 other airports, and each of the 200 packages can be either unloaded (if it is loaded) or loaded into any plane at its airport (if it is unloaded).
- So in any state there is a minimum of 450 actions (when all the packages are at airports with no planes) and a maximum of 10,450 (when all packages and planes are at the same airport). On average, let's say there are about 2000 possible actions per

state, so the search graph up to the depth of the obvious solution has about 2000 [1] nodes.

- Thus, uninformed forward search is infeasible. The key is using accurate heuristics, which can often be derived automatically, making forward search practical.

4.2.2 Backward (regression) relevant-states search

- In regression search we start at the goal and apply the actions backward until we find a sequence of steps that reaches the initial state. It is called relevant-states search because we only consider actions that are relevant to the goal (or current state).
- We start with the goal, which is a conjunction of literals forming a description of a set of states—for example, the goal ¬Poor ∧Famous describes those states in which Poor is false, Famous is true, and any other fluent can have any value. If there are n ground fluents in a domain, then there are 2^n ground states (each fluent can be true or false), but 3^n descriptions of sets of goal states (each fluent can be positive, negative, or not mentioned).
- In general, backward search works only when we know how to regress from a state description to the predecessor state description.
- For example, it is hard to search backwards for a solution to the n-queens problem because there is no easy way to describe the states that are one move away from the goal. Happily, the PDDL representation was designed to make it easy to regress actions—if a domain can be expressed in PDDL, then we can do regression search on it. Given a ground goal description g and a ground action a, the regression from g over a gives us a state description g
    - $g' = (g - \text{ADD}(a)) \cup \text{Precond}(a)$
- In regression, ADD(a) is removed since its effects need not hold before the action, while Precond(a) must hold. DEL(a) is ignored because we can't know if those fluents were true beforehand.
- To get the full advantage of backward search, we need to deal with partially uninstantiated actions and states, not just ground ones. For example, suppose the goal is to deliver a specific piece of cargo to SFO: $\text{At}(C_2, \text{SFO})$. That suggests the action
- Note that we have standardized variable names (changing p to p' in this case) so that there will be no confusion between variable names if we happen to use the same action schema twice in a plan.
- **Example:** goal $\text{At}(C_2, \text{SFO})$ regresses via Unload(C2, p', SFO), giving

$$g' = \text{In}(C_2, p') \wedge \text{At}(p', \text{SFO}) \wedge \text{Cargo}(C_2) \wedge \text{Plane}(p') \wedge \text{Airport}(\text{SFO}).$$

- Forward search uses applicable actions, while backward search uses relevant actions (those that could achieve the goal).
- In backward search, actions are relevant if:
    - At least one effect unifies with a goal literal.
    - No effect negates a goal literal.

- **Example:** goal At(C_2, SFO) → relevant actions are Unload(C_2, p', SFO) (plane unspecified or specific). Using the most general unifier reduces branching.
- Another example: goal Own(0136042597) → relevant action is Buy(0136042597), not all 10 billion buys.
- To achieve At(C_2, SFO), before unloading we must have:
    - Cargo C_2 inside some plane p,
    - Plane p' at SFO,
    - And facts that C_2 is cargo, p' is a plane, SFO is an airport.
- In short: $g' = In(C_2, p') \wedge At(p', SFO) \wedge Cargo(C_2) \wedge Plane(p') \wedge Airport(SFO)$ is the predecessor condition for unloading.
- Forward search → choose applicable actions (can be executed now).
- Backward search → choose relevant actions (could achieve the current goal).
- An action is relevant if:
    - At least one effect unifies with a goal literal.
    - No effect negates a goal literal.
- **Example:** goal At(C_2, SFO) → relevant action is Unload(C_2, p', SFO) (plane can be unspecified using the most general unifier to cut branching).
- Backward search uses relevant actions that achieve but don't conflict with the goal.
- As another example, consider the goal Own(0136042597), given an initial state with 10 billion ISBNs, and the single action schema
  A = Action(Buy(i), PRECOND:ISBN (i), EFFECT:Own(i)) .
- Backward search avoids huge branching (e.g., instead of 10B Buy actions, unify goal Own(0136042597) with effect Own(i) → regress to ISBN(0136042597), which holds in the initial state).
- Formally: an action a is relevant to goal g if some effect unifies with a goal literal (via substitution θ) and no effect negates any goal literal. Backward search reduces branching but makes heuristic design harder, so most systems prefer forward search.

4.2.3 Heuristics for planning

- Heuristics estimate the distance from a state s to the goal (h(s)). An admissible heuristic never overestimates and allows A* to find optimal solutions.
- An **admissible heuristic** can be derived by defining a relaxed problem that is easier to solve.
- Think of a search problem as a graph where the nodes are states and the edges are actions. The problem is to find a path connecting the initial state to a goal state.
- There are two ways we can relax this problem to make it easier:
    - by adding more edges to the graph, making it strictly easier to find a path,
    - or by grouping multiple nodes together, forming an abstraction of the state space that has fewer states, and thus is easier to search
- For example, the ignore preconditions heuristic drops all preconditions from actions. Every action becomes applicable in every state, and any single goal fluent can be

achieved in one step (if there is an applicable action—if not, the problem is impossible).

- This almost implies that the number of steps required to solve the relaxed problem is the number of unsatisfied goals—almost but not quite, because
  (1) some action may achieve multiple goals and
  (2) some actions may undo the effects of others.
- For many problems an accurate heuristic is obtained by considering
  (1) and ignoring
  (2). First, we relax the actions by removing all preconditions and all effects except those that are literals in the goal.
- Then, we count the minimum number of actions required such that the union of those actions' effects satisfies the goal. This is an instance of the set-cover problem. There is one minor irritation: the set-cover problem is NP-hard.
- The minimum number of actions required such that the union of those actions' effects satisfies the goal. This is an instance of the **set-cover problem.**
- Fortunately a simple greedy algorithm is guaranteed to return a set covering whose size is within a factor of log n of the true minimum covering, where n is the number of literals in the goal. Unfortunately, the greedy algorithm loses the guarantee of admissibility (A heuristic is **admissible** if it never overestimates the true cost to reach the goal).

  Sliding block puzzle(8 puzzle)

  Action(Slide(t, s_1, s_2),

  PRECOND:On(t, s_1) ∧ Tile(t) ∧ Blank (s_2) ∧ Adjacent (s_1, s_2)

  EFFECT:On(t, s_2) ∧ Blank(s_1)∧ ¬On(t, s_1) ∧¬Blank(s_2))
- The subgoal independence assumption can be optimistic or Pessimistic.
- In **optimistic** when there are negative interactions between the subplans for each of goal.
- For example when an action in one subplan deletes a goal achieved by another subgoal.
- In **Pessimistic** and therefore at inadmissible, when subplan deletes a goal contain redundant actions- for instance, two actions that could be replaced by a single action in the merged plan
- Heuristics in planning can be made by ignoring some action preconditions.
- Ignoring more gives simpler heuristics (like misplaced tiles).Ignoring fewer gives stronger heuristics (like Manhattan distance).
- Heuristics in planning can also be made by ignoring delete lists (negative effects). This makes progress towards goals monotonic (no action undoes another), so hill-climbing works without dead ends, though the problem is still hard in theory.
- Another approach is state abstraction, where we ignore some details (fluents) to reduce the number of states. Abstract solutions are shorter but still valid heuristics, and they can be expanded back into full solutions.

- In simple terms:

- Ignore delete lists → easier progress, good heuristics.
- State abstraction → fewer states, admissible heuristics**.**

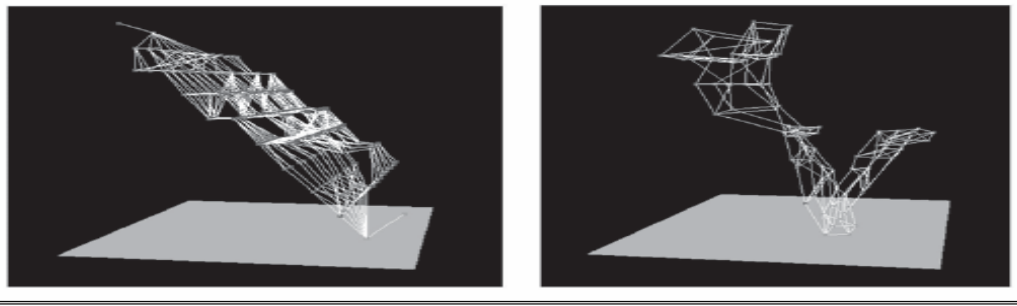


Figure 10.6 Two state spaces from planning problems with the ignore-delete-lists heuristic. The height above the bottom plane is the heuristic score of a state; states on the bottom plane are goals. There are no local minima, so search for the goal is straightforward. From Hoffmann (2005).

- A key heuristic idea is **decomposition:** split goals into subgoals, solve them separately, then combine costs.
  - Max of subgoal costs → always admissible but often too low.
  - Sum of costs → better estimate, but only admissible if subgoals are independent.
- Other techniques:
  - Planning graphs and pattern databases improve estimates.
  - FF (FastForward) uses the ignore-delete-lists heuristic with planning graphs, plus hill-climbing search; if stuck on a plateau, it switches to iterative deepening or restarts.
- Decomposition + abstraction + smart heuristics (like in FF) make planning search far more efficient.

4.3 Planning Graphs

- A special data structure called a **planning graph** can be used to give better heuristic estimates. These heuristics can be applied to any of the search techniques
- we can search for a solution over the space formed by the planning graph, using an algorithm called GRAPH PLAN.
- A planning graph is a directed graph organized into **levels**: first a level S_0 for the initial state, consisting of nodes representing each fluent that holds in S_0 ,then a level A_0 consisting of nodes for each ground action that might be applicable in S_0 then alternating levels S_i followed by A_i until we reach a termination condition.

$Init(Have(Cake))$
$Goal(Have(Cake) \wedge Eaten(Cake))$
$Action(Eat(Cake)$
PRECOND: $Have(Cake)$
EFFECT: $\neg Have(Cake) \wedge Eaten(Cake))$
$Action(Bake(Cake)$
PRECOND: $\neg Have(Cake)$
EFFECT: $Have(Cake))$

Figure 10.7 The "have cake and eat cake too" problem.

- Figure 10.7 shows a simple planning problem, and Figure 10.8 shows its planning graph. Each action at level A_i is connected to its preconditions at S_i and its effects at S_{i+1}.
- So a literal appears because an action caused it, but we also want to say that a literal can persist if no action negates it. This is represented by a persistence action (sometimes called a no-op).
- For every literal C, we add to the problem a persistence action with precondition C and effect C. Level A_0 in Figure 10.8 shows one "real" action, Eat (Cake), along with two persistence actions drawn as small square boxes.

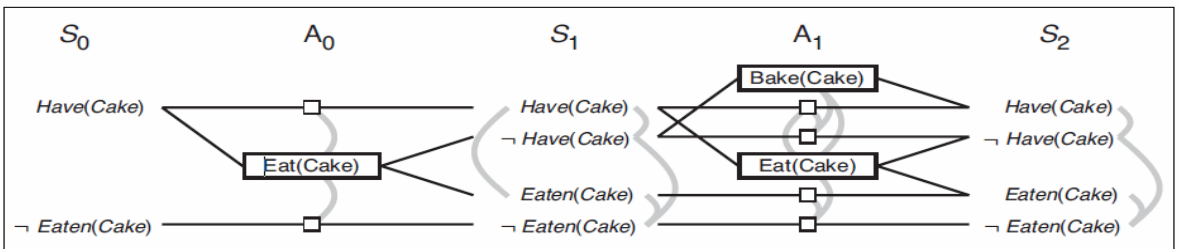


Figure 10.8 The planning graph for the "have cake and eat cake too" problem up to level S_2. Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i, then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

- Level A_0 contains all the actions that could occur in state S_0, but just as important it records conflicts between actions that would prevent them from occurring together. The gray lines in Figure 10.8 indicate mutual exclusion (or mutex) links.
- For example, Eat (Cake) is mutually exclusive with the persistence of either Have(Cake) or ¬Eaten(Cake).
- Level S_1 contains all the literals that could result from picking any subset of the actions in A_0, as well as mutex links (gray lines) indicating literals that could not appear together, regardless of the choice of actions.
- For example, Have(Cake) and Eaten(Cake) are mutex: depending on the choice of actions in A_0, either, but not both, could be the result. In other words, S_1 represents a belief state: a set of possible states. The members of this set are all subsets of the literals such that there is no mutex link between any members of the subset.

- We continue in this way, alternating between state level S_i and action level A_i until we reach a point where two consecutive levels are identical. At this point, we say that the graph has **leveled off**.
- Persistence action(called a no-op)
- A mutex relation holds between two actions at a given level if any of the following three conditions holds:
- **Inconsistent effects:** one action negates an effect of the other.

Example: Eat (Cake) and the persistence of Have(Cake) have inconsistent effects because they disagree on the effect Have(Cake).

- **Interference:** one of the effects of one action is the negation of a precondition of the other.

Example: Eat (Cake) interferes with the persistence of Have(Cake) by negating its precondition.

- **Competing needs:** one of the preconditions of one action is mutually exclusive with a precondition of the other.

Example: Bake(Cake) and Eat (Cake) are mutex because they compete on the value of the Have(Cake) precondition

4.2.1 Planning graphs for heuristic estimation

- A planning graph, once constructed, is a rich source of information about the problem.
    - **First**, if any goal literal fails to appear in the final level of the graph, then the problem is unsolvable.
    - **Second**, we can estimate the cost of achieving any goal literal g_i from state s as the level at which g_i first appears in the planning graph constructed from initial state s. we call this the level cost of g_i.
- Heuristic counts just the level and not the number of actions. For this reason, it is common to use a **serial planning graph** for computing heuristics. A serial graph insists that only one action can actually occur at any given time step; this is done by adding mutex links between every pair of nonpersistence actions. Level costs extracted from serial graphs are often quite reasonable estimates of actual costs.
- To estimate the cost of a conjunction of goals, there are three simple approaches. The
- **Max-level** heuristic simply takes the maximum level cost of any of the goals; this is admissible, but not necessarily accurate.
- The **Level sum heuristic**, following the subgoal independence assumption, returns the sum of the level costs of the goals; this can be inadmissible but works well in practice for problems that are largely decomposable
- The **Set-level** heuristic finds the level at which all the literals in the conjunctive goal appear in the planning graph without any pair of them being mutually exclusive.
- A planning graph cannot recognize some unsolvable problems, such as the blocks-world goal of putting A on B, B on C, and C on A. Each pair of subgoals is

achievable, but all three together are mutually inconsistent. Since the graph only records pairwise mutexes, it misses this global inconsistency. Detecting impossibility requires explicit search over the planning graph.

4.2.2 The GRAPHPLAN algorithm

```
function GRAPHPLAN(problem) returns solution or failure
  graph ← INITIAL-PLANNING-GRAPH(problem)
  goals ← CONJUNCTS(problem.GOAL)
  nogoods ← an empty hash table
  for tl = 0 to ∞ do
    if goals all non-mutex in S_t of graph then
      solution ← EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)
      if solution ≠ failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph ← EXPAND-GRAPH(graph, problem)
```

Figure 10.9 The GRAPHPLAN algorithm. GRAPHPLAN calls EXPAND-GRAPH to add a level until either a solution is found by EXTRACT-SOLUTION, or no solution is possible.

- The GRAPHPLAN algorithm (Figure 10.9) repeatedly adds a level to a planning graph with EXPAND-GRAPH. Once all the goals show up as nonmutex in the graph, GRAPHPLAN calls EXTRACT-SOLUTION to search for a plan that solves the problem. If that fails, it expands another level and tries again, terminating with failure when there is no reason to go on.

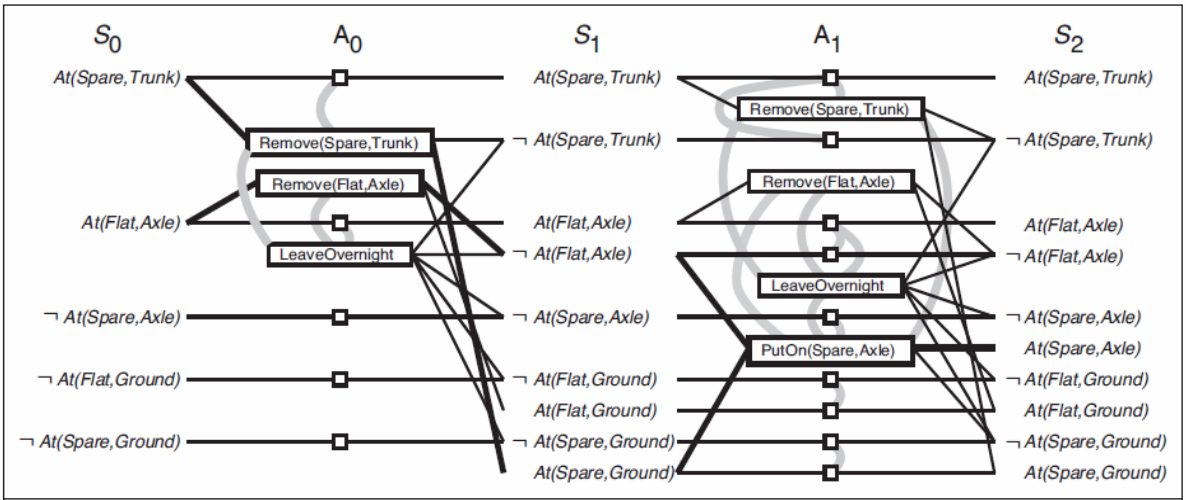


Figure 10.10 The planning graph for the spare tire problem after expansion to level S_2. Mutex links are shown as gray lines. Not all links are shown, because the graph would be too cluttered if we showed them all. The solution is indicated by bold lines and outlines.

some of the examples of mutex relations and their causes:

- **Inconsistent effects:** Remove(Spare, Trunk) is mutex with LeaveOvernight because one has the effect At(Spare, Ground) and the other has its negation.
- **Interference:** Remove(Flat, Axle) is mutex with LeaveOvernight because one has the precondition At(Flat, Axle) and the other has its negation as an effect.

- **Competing needs:** PutOn(Spare, Axle) is mutex with Remove(Flat, Axle) because one has At(Flat, Axle) as a precondition and the other has its negation.
- **Inconsistent support:** At(Spare, Axle) is mutex with At(Flat, Axle) in S2 because the only way of achieving At(Spare, Axle) is by PutOn(Spare, Axle), and that is mutex with the persistence action that is the only way of achieving At(Flat, Axle). Thus, the mutex relations detect the immediate conflict that arises from trying to put two objects in the same place at the same time.
- EXTRACT-SOLUTION as a backward search problem, where each state in the search contains a pointer to a level in the planning graph and a set of unsatisfied goals. We define this search problem as follows:
    - The initial state is the last level of the planning graph, S_n, along with the set of goals from the planning problem.
    - The actions available in a state at level S_i are to select any conflict-free subset of the actions in A_{i-1} whose effects cover the goals in the state. The resulting state has level S_{i-1} and has as its set of goals the preconditions for the selected set of actions. By "conflict free," we mean a set of actions such that no two of them are mutex and no two of their preconditions are mutex.
    - The goal is to reach a state at level S_0 such that all the goals are satisfied.
    - The cost of each action is 1.

4.2.3 Termination of GRAPHPLAN

- GRAPHPLAN will in fact terminate and return failure when there is no solution. The first thing to understand is why we can't stop expanding the graph as soon as it has leveled off. At that point, if no plan exists, we can safely terminate with failure. This works because planning graphs have monotonic properties:
    - **Literals increase monotonically in a planning graph**: once a literal appears at some level, persistence actions guarantee it remains present at all later levels. Thus, literals can only be added, never removed.
    - **Actions increase monotonically**: once an action is possible at some level, it remains possible at all later levels, because its preconditions (literals) never disappear.
    - **Mutexes decrease monotonically:** If two actions are mutex at a given level A_i, then they will also be mutex for all previous levels at which they both appear. The same holds for mutexes between literals. It might not always appear that way in the figures, because the figures have a simplification: We can see that "mutexes decrease monotonically" is true if you consider that these invisible literals and actions are mutex with everything

4.4 Analysis of Planning Approaches

- Planning combines the two major areas of AI we have covered so far: search and logic. A planner can be seen either as a program that searches for a solution or as one that (constructively) proves the existence of a solution.
- This synergy has greatly improved planner performance and industrial adoption, but we still lack a clear guide for which techniques suit which problems, leaving room for new dominant methods to emerge.
- Planning is foremost an exercise in controlling combinatorial explosion. If there are n propositions in a domain, then there are 2^n states and is PSPACE-hard.
- Speedups come from decomposing problems into independent subproblems, but negative interactions break decomposability.
- Graphplan handles this by recording mutexes (mutual exclusions). SATPlan encodes them in CNF form. Forward search uses heuristics to find useful patterns, working even when subproblems aren't fully independent.
- Sometimes it is possible to solve a problem efficiently by recognizing that negative interactions can be ruled SERIALIZABLE out. We say that a problem has serializable subgoals if there exists an order of subgoals such that the planner can achieve them in that order without having to undo any of the previously achieved subgoals.
- **For example**, in the blocks world, if the goal is to build a tower (e.g., A on B, which in turn is on C, which in turn is on the Table, as in Figure 10.4 on page 371), then the subgoals are serializable bottom to top: if we first achieve C on Table, we will never have to undo it while we are achieving the other subgoals.
- A planner that uses the bottom-to-top trick can solve any problem in the blocks world without backtracking (although it might not always find the shortest plan).
- As a more complex example, for the Remote Agent planner that commanded NASA's Deep Space One spacecraft, it was determined that the propositions involved in commanding a spacecraft are serializable.
- This is perhaps not too surprising, because a spacecraft is designed by its engineers to be as easy as possible to control (subject to other constraints).
- Taking advantage of the serialized ordering of goals, the Remote Agent planner was able to eliminate most of the search. This meant that it was fast enough to control the spacecraft in real time, something previously considered impossible.
- Planners such as GRAPHPLAN, SATPLAN, and FF have moved the field of planning forward, by raising the level of performance of planning systems, by clarifying the representational and combinatorial issues involved, and by the development of useful heuristics.
- However, there is a question of how far these techniques will scale. It seems likely that further progress on larger problems cannot rely only on factored and propositional representations, and will require some kind of synthesis of first-order and hierarchical representations with the efficient heuristics currently in use.

4.5 Hierarchical Planning

- Traditional AI planning methods use fixed, low-level atomic actions, real-world tasks—like human behavior—require planning at multiple levels of abstraction because of the immense number of actions involved.
- Humans simplify complex tasks by **hierarchical decomposition**: breaking a high-level goal (e.g., "go on vacation") into smaller subgoals ("go to airport," "fly to Hawaii," etc.), each of which can be further refined into detailed actions until they can be executed directly.
- This hierarchical planning allows complex problems to be managed efficiently, since each level deals with only a few sub-tasks. In contrast, non-hierarchical methods that plan directly at the atomic action level are computationally impractical for large problems.

4.5.1 High level actions

- Hierarchical Task Network (HTN) planning, a formalism for hierarchical decomposition in AI.
- In HTN planning:
    - Primitive actions are basic, executable steps with defined preconditions andeffects.
    - High-Level Actions (HLAs) represent abstract tasks (e.g., Go(Home, SFO)) that can be refined into sequences of lower-level actions.
    - A refinement may include other HLAs or primitive actions; when it consists only of primitive actions, it's called an implementation.
- A high-level plan achieves the goal if at least one of its implementations leads to success—since the agent can choose which refinement to execute. This differs from nondeterministic planning, where all outcomes must succeed because the agent has no control over which one occurs.
- If a High-Level Action (HLA) has only one implementation, its preconditions and effects can be derived directly from that implementation, allowing it to be treated like a primitive action.
- Using well-chosen HLAs can reduce search complexity dramatically—from exponential to linear in plan depth—though creating such HLAs is challenging.
- When HLAs have multiple implementations, planners can either:
    - Search among implementations to find one that works, or
    - Reason abstractly about HLAs without expanding them, allowing the creation of provably correct abstract plans without examining every detailed implementation.

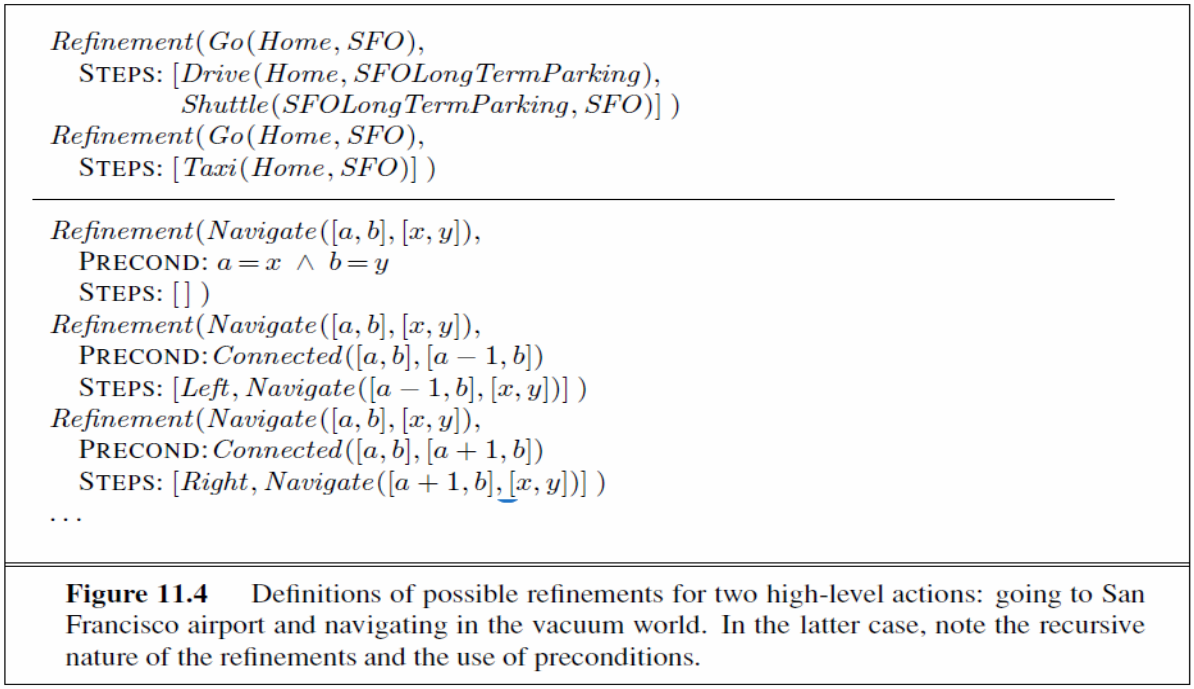


Figure 11.4 Definitions of possible refinements for two high-level actions: going to San Francisco airport and navigating in the vacuum world. In the latter case, note the recursive nature of the refinements and the use of preconditions.

- A refinement defines how a high-level (abstract) action can be implemented as a sequence of lower-level (more concrete) actions. This process is also called task decomposition.
- This high-level goal means: "Go from Home to San Francisco Airport."There are two possible refinements (alternative ways) to achieve this:
- Here, the plan is broken into:
    - Drive from Home to the SFO Long-Term Parking,
    - then take a Shuttle from parking to the SFO terminal.
- Another, simpler plan — just take a taxi directly from Home to SFO.
- **Interpretation:** The planner can choose either refinement, depending on conditions (e.g., cost, availability, or constraints).So, Go(Home, SFO) is abstract, while Drive, Shuttle, and Taxi are concrete actions.
- Second Example: Navigate([a, b], [x, y])This represents navigation in the vacuum world (a grid-based environment), where the agent moves from location [a,b] to [x,y].
- There are recursive refinements, since navigating often requires multiple smaller moves.
- If the current position [a,b] equals the goal [x,y],→ do nothing (STEPS: []).
- If there's a connection to the left, move Left, then recursively Navigate from [a-1,b] to [x,y]
- Similarly, if connected to the right, move Right, then recursively Navigate again.
- **Interpretation:** This recursive structure allows the agent to build a full sequence of moves (Left/Right/Up/Down, etc.) to reach the goal.

Concept	Meaning
Refinement	A decomposition of a high-level action into smaller steps.
Preconditions	Conditions that must hold for the refinement to be valid.
Recursive Refinement	Some actions (like navigation) are broken down repeatedly until a base condition is reached.
Alternative Refinements	Multiple ways to achieve the same goal (e.g., taxi vs. drive+shuttle).

4.5.2 Searching for primitive solutions

- In Hierarchical Task Network (HTN) planning, a single top-level action called Act represents the overall goal. The planner refines Act step by step into smaller actions until a complete plan is formed.
- Classical planning can be represented in this framework by defining:
    - For each primitive action ai, a refinement of Act with steps [ai, Act], and
    - A stopping rule: a refinement with no steps when the goal condition is already satisfied.
- The algorithm repeatedly picks a high-level action (HLA) in the plan and replaces it with one of its refinements, continuing until the plan achieves the goal. It can use breadth-first, depth-first, or iterative-deepening search.
- HTN planning efficiently explores structured plans that reflect domain knowledge through refinements and preconditions. Systems like O-PLAN have demonstrated its power — generating large-scale industrial production schedules involving thousands of operations and millions of steps.
- Because of their hierarchical structure, HTN plans are both powerful and easy for humans to interpret.
- HTN planning greatly reduces computational cost compared to flat (non-hierarchical) planning.
- If a non-hierarchical planner has cost $O(b^d)$ for d actions and branching factor b, an HTN planner with r refinements per action and k subactions per refinement can generate only r^((d−1)/(k−1)) possible decomposition trees — much smaller when r is small and k is large.
- Thus, using a few refinements that each produce longer, reusable action sequences yields huge savings. HTN efficiency depends on a library of methods—predefined or learned from experience—that describe how to achieve high-level goals. Over time, the planner can learn and generalize these methods, becoming more competent and efficient.

```
function HIERARCHICAL-SEARCH(problem, hierarchy) returns a solution, or failure
  frontier ← a FIFO queue with [Act] as the only element
  loop do
    if EMPTY?(frontier) then return failure
    plan ← POP(frontier)  /* chooses the shallowest plan in frontier */
    hla ← the first HLA in plan, or null if none
    prefix,suffix ← the action subsequences before and after hla in plan
    outcome ← RESULT(problem.INITIAL-STATE, prefix)
    if hla is null then  /* so plan is primitive and outcome is its result */
      if outcome satisfies problem.GOAL then return plan
    else for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
      frontier ← INSERT(APPEND(prefix, sequence, suffix), frontier)
```

Figure 11.5 A breadth-first implementation of hierarchical forward planning search. The initial plan supplied to the algorithm is [*Act*]. The REFINEMENTS function returns a set of action sequences, one for each refinement of the HLA whose preconditions are satisfied by the specified state, *outcome*.

4.5.3 Searching for abstract solutions

- In hierarchical planning, it's inefficient to refine every high-level action (HLA) down to primitive steps just to verify a plan. Instead, we can assign precondition–effect descriptions to HLAs, allowing reasoning at a higher level.
- For this to work correctly, the **downward refinement property** must hold — meaning any high-level plan that appears to achieve the goal must have at least one concrete implementation that truly does.
- Writing such HLA descriptions is simple when each HLA has one implementation but tricky when there are multiple. Including only effects common to all implementations is too conservative, since the agent—not an adversary—chooses which refinement to use. Thus, we need smarter ways to describe HLAs that preserve correctness without over-restricting them.

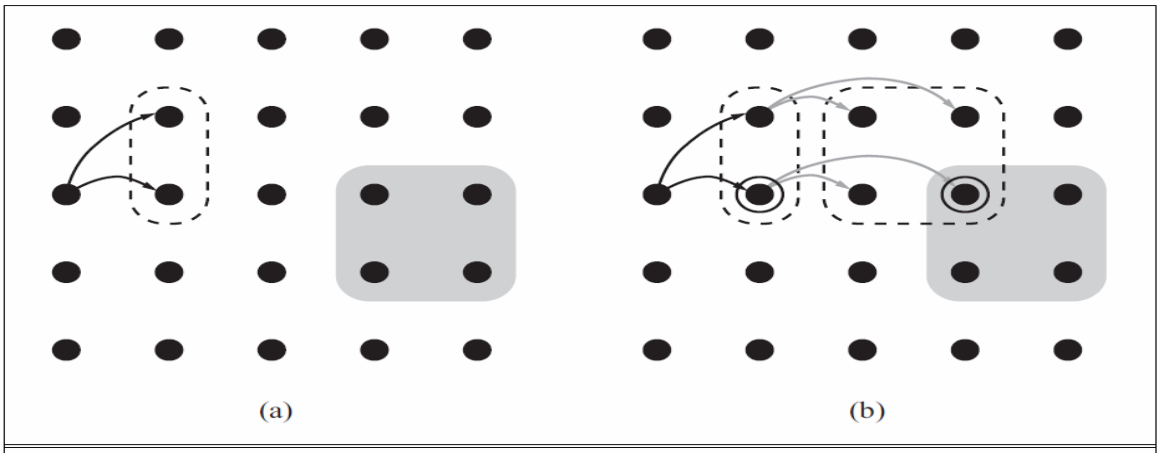


Figure 11.6 Schematic examples of reachable sets. The set of goal states is shaded. Black and gray arrows indicate possible implementations of h_1 and h_2, respectively. (a) The reachable set of an HLA h_1 in a state s. (b) The reachable set for the sequence $[h_1, h_2]$. Because this intersects the goal set, the sequence achieves the goal.

- When we plan hierarchically (using HLAs — High-Level Actions), each HLA can have multiple possible refinements (implementations).
- For instance,

  "Go(Home, SFO)" can be done via taxi or driving yourself — two different refinements.
- But this leads to nondeterminism — we don't know which refinement will happen until execution.

Demonic Vs Angelic Non determinism

Concept	Who Makes the Choice?	Interpretation
Demonic nondeterminism	An *adversary (demon)* chooses the worst-case outcome.	The agent must succeed **no matter which implementation** occurs.
Angelic nondeterminism	The *agent* chooses the best-case outcome.	The agent can assume it will choose the best implementation to achieve the goal.

Reachable Sets

- We define REACH(s, h) = the set of all states reachable from state s by executing any implementation of HLA h.
  - For a single HLA h, it's all the possible resulting states.
  - For a sequence of HLAs [h , h]:

$$\text{REACH}(s, [h_1, h_2]) = \bigcup_{s' \in \text{REACH}(s, h_1)} \text{REACH}(s', h_2)$$

- So the reachable set of a sequence is the union of reachable states after applying each HLA in turn.
- When Does a High-Level Plan "Achieve" the Goal?
- A plan (sequence of HLAs) achieves the goal if:
  - (,[1, 2,…])∩ ≠∅
- That is, at least one reachable state satisfies the goal.
- Angelic condition: Some implementation works.
- Demonic condition: All implementations must work.

Example with Angelic Effects

We can summarize the possible effects on a variable (fluent):

Start Value	Always True	Always False	Choice
True	keep True	make False	agent can choose
False	keep False	make True	agent can choose

Hence, there are 9 possible combinations (3 × 3).

Notation:

∇ A → possibly add A

∇ A → possibly delete A

∇±A → possibly add or delete A

Example:

Go(Home, SFO) possibly deletes Cash (if taxi taken)

So effect = ∇ Cash.

Sample HLA Definitions

Action(h1, PRECOND: ¬A, EFFECT: A ∧ ∇ B)

Action(h2, PRECOND: ¬B, EFFECT: ∇ A ∧ ∇±C)

If the initial state has only B = true and goal is A ∧ C,

then sequence [h1, h2] achieves the goal under angelic semantics:

- Choose h implementation that makes B false and adds A.
- Choose h implementation that keeps A true and makes C true.

Thus, the reachable set includes a goal state.

Approximation of Reachable Sets

In practice, it's often impossible to compute exact reachable sets — too complex or infinite.

So we use approximations:

Type	Meaning	Relation
Optimistic (REACH^+)	Overestimates reachable states (may include impossible ones).	$\text{REACH}(s,h) \subseteq \text{REACH}^+(s,h)$
Pessimistic (REACH^-)	Underestimates reachable states (may miss some).	$\text{REACH}^-(s,h) \subseteq \text{REACH}(s,h)$

For example:

Optimistic: "Go(Home, SFO)" may possibly delete Cash and possibly add At(Car, SFO_LTP).

Realistically, you can't do both simultaneously — but the optimistic description includes both.

- Demonic nondeterminism = worst-case, agent must handle all outcomes.
- Angelic nondeterminism = best-case, agent can pick the successful refinement.
- Reachable sets represent all states achievable via HLAs.
- Plan success (angelic) = reachable set intersects goal states
- Optimistic / pessimistic reachable sets allow approximate planning when exact sets are infeasible.

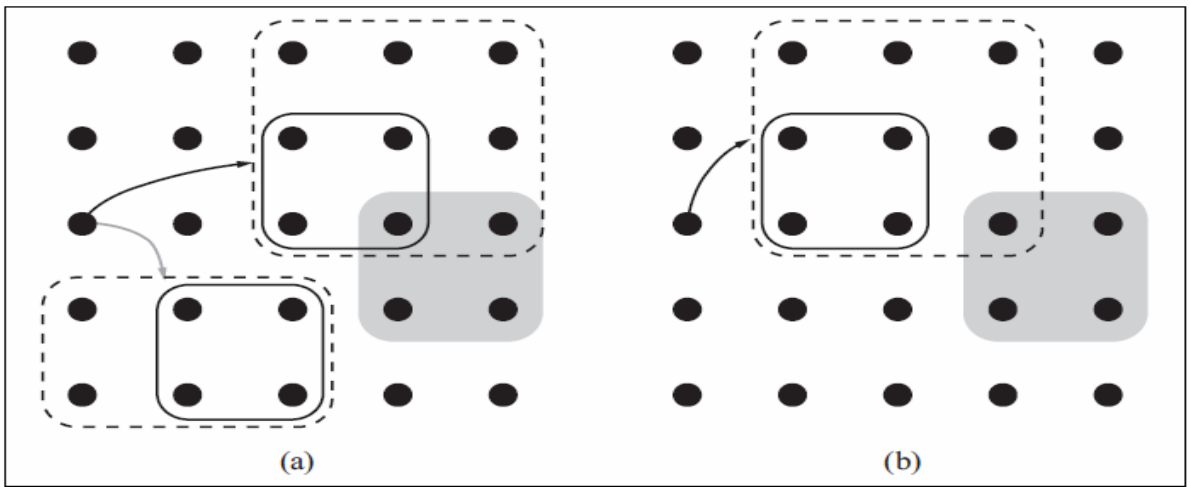


Figure 11.7 Goal achievement for high-level plans with approximate descriptions. The set of goal states is shaded. For each plan, the pessimistic (solid lines) and optimistic (dashed lines) reachable sets are shown. (a) The plan indicated by the black arrow definitely achieves the goal, while the plan indicated by the gray arrow definitely doesn't. (b) A plan that would need to be refined further to determine if it really does achieve the goal.

Earlier, we saw that each High-Level Action (HLA) can have:

- a **pessimistic reachable set** $REACH^{-}(s, h)$ (definitely reachable states), and
- an **optimistic reachable set** $REACH^{+}(s, h)$ (possibly reachable states).

Relationship:

$$REACH^{-}(s, h) \subseteq REACH(s, h) \subseteq REACH^{+}(s, h)$$

So:

Case	Interpretation
$REACH^{-} \cap GOAL \neq \emptyset$	✅ The plan **definitely works**
$REACH^{+} \cap GOAL = \emptyset$	❌ The plan **definitely fails**
$REACH^{-} \cap GOAL = \emptyset$ but $REACH^{+} \cap GOAL \neq \emptyset$	🤔 **Uncertain** — may or may not work → **refine plan**

Example:

- Imagine a vacation plan:
  - "Spend two days on each of seven islands."
- Optimistically, that might sound feasible — the reachable set includes all seven islands.But pessimistically, considering travel time and logistics, it may not intersect the "relaxation goal."
- Therefore, you refine the plan to include transportation details.That's how angelic planning mimics human reasoning — abstract first, refine later.

The ANGELIC-SEARCH Algorithm

This is a breadth-first hierarchical search that:

- uses optimistic/pessimistic reachable sets to decide early whether a high-level plan can succeed or fail,
- and commits only to plans that are guaranteed to work.

```
function ANGELIC-SEARCH(problem, hierarchy, initialPlan) returns solution or fail
  frontier ← a FIFO queue with initialPlan as the only element
  loop do
    if EMPTY?(frontier) then return fail
    plan ← POP(frontier)  /* chooses the shallowest node in frontier */
    if REACH+(problem.INITIAL-STATE, plan) intersects problem.GOAL then
      if plan is primitive then return plan  /* REACH+ is exact for primitive plans */
      guaranteed ← REACH−(problem.INITIAL-STATE, plan) ∩ problem.GOAL
      if guaranteed ≠ { } and MAKING-PROGRESS(plan, initialPlan) then
        finalState ← any element of guaranteed
        return DECOMPOSE(hierarchy, problem.INITIAL-STATE, plan, finalState)
      hla ← some HLA in plan
      prefix, suffix ← the action subsequences before and after hla in plan
      for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
        frontier ← INSERT(APPEND(prefix, sequence, suffix), frontier)

function DECOMPOSE(hierarchy, s0, plan, sf) returns a solution
  solution ← an empty plan
  while plan is not empty do
    action ← REMOVE-LAST(plan)
    si ← a state in REACH−(s0, plan) such that sf ∈ REACH−(si, action)
    problem ← a problem with INITIAL-STATE = si and GOAL = sf
    solution ← APPEND(ANGELIC-SEARCH(problem, hierarchy, action), solution)
    sf ← si
  return solution
```

Figure 11.8 A hierarchical planning algorithm that uses angelic semantics to identify and commit to high-level plans that work while avoiding high-level plans that don't. The predicate MAKING-PROGRESS checks to make sure that we aren't stuck in an infinite regression of refinements. At top level, call ANGELIC-SEARCH with $[Act]$ as the *initialPlan*.

Key Idea: Computational Advantage

Algorithm	How It Works	Efficiency
BREADTH-FIRST-SEARCH	Expands all primitive actions	Exponential (e.g., (5^d))
HIERARCHICAL-SEARCH	Uses hierarchy but still explores many refinements	Still exponential
ANGELIC-SEARCH	Prunes entire high-level branches early (based on reachable sets)	**Almost linear**

Example:
Vacuum world with rooms & corridors:

- ❑ HLAs: Navigate and CleanWholeRoom
- ❑ Angelic-Search can plan room by room, pruning irrelevant cleaning orders.

Humans also think this way — we commit to high-level steps ("clean each room") without detailing "which tile first" until needed.

Cost-Based Angelic Planning

We can generalize reachable sets to costed reachable sets:

- Each state has a minimum cost to reach.
- Use optimistic & pessimistic bounds on cost.

Then:

- Angelic Search can find provably optimal abstract plans without expanding all refinements.
- Useful for hierarchical lookahead and online planning (like LRTA^*).
- Intuitive Analogy: “Planning a Hawaii Trip”

Planning Level	Description	Stage
Abstract	“Spend two days on each island.”	Initial Angelic Plan
Mid-Level	Add travel days between islands.	Refinement
Detailed	Book specific flights, hotels.	Final Subplans
Execution	Decide which beach to visit that day.	Primitive actions

4.6 Multi-Agent Planning

- When multiple agents exist in an environment, each faces a multiagent planning problem, trying to achieve its own goals—sometimes helping, sometimes hindering others.
- Between single-agent and full multiagent planning lies a spectrum:
    - **Multieffector planning:** One agent controls several effectors (e.g., typing and speaking simultaneously).
    - **Multibody planning:** Effectors are physically separate (e.g., multiple robots) but share information and a common plan.
    - **Decentralized planning:** Communication limits prevent full information sharing; each body follows its own subplan with possible communication actions.
- If a single planner controls all bodies, there is **one shared goal**.
- If distinct agents plan independently but share a goal (e.g., tennis doubles), **coordination** is required to avoid conflicts.
- When agents have **different goals** (e.g., opposing teams), the situation becomes **competitive or zero-sum.**
- Many real systems mix both styles—centralized offline planning with autonomous execution (e.g., delivery fleets).
- **Incentives** like salaries align individual goals with organizational objectives, confirming a true multiagent system.

4.6.1 Planning with multiple simultaneous actions

- In multiactor settings (covering multieffector, multibody, and multiagent cases), all entities are called actors.
- The goal is to define:
    - Transition models (how joint actions change the world),
    - Correct plans (those that achieve the goal when executed by all actors), and
    - Efficient planning algorithms for such systems.

- Assuming perfect synchronization, all actions take equal time and occur simultaneously in each step of the joint plan.

$Actors(A, B)$
$Init(At(A, LeftBaseline) \wedge At(B, RightNet) \wedge$
$\quad Approaching(Ball, RightBaseline)) \wedge Partner(A, B) \wedge Partner(B, A)$
$Goal(Returned(Ball) \wedge (At(a, RightNet) \vee At(a, LeftNet))$
$Action(Hit(actor, Ball),$
$\quad$ PRECOND: $Approaching(Ball, loc) \wedge At(actor, loc)$
$\quad$ EFFECT: $Returned(Ball))$
$Action(Go(actor, to),$
$\quad$ PRECOND: $At(actor, loc) \wedge to \neq loc,$
$\quad$ EFFECT: $At(actor, to) \wedge \neg At(actor, loc))$

Figure 11.10 The doubles tennis problem. Two actors A and B are playing together and can be in one of four locations: *LeftBaseline*, *RightBaseline*, *LeftNet*, and *RightNet*. The ball can be returned only if a player is in the right place. Note that each action must include the actor as an argument.

- In multiactor planning, the transition model extends from single actions to joint actions a , …, a , where each actor chooses an action. This creates two challenges:
    - The transition model must describe b^n joint actions.
    - The branching factor becomes exponential in the number of actors.
- To manage complexity, researchers aim to decouple actors, so complexity grows roughly linearly with n.
- If actors are independent, each can be planned for separately. For loosely coupled actors, problems are treated as independent first, then interactions are corrected.
- For example, in a tennis doubles scenario:
    - A simple joint plan works if only one player hits the ball.
    - Problems arise when both hit simultaneously — real-world constraints aren't captured.
- This is solved using concurrent action lists, specifying actions that must or must not occur together.

Example:

$$Action(Hit(a, Ball),$$
$$\text{CONCURRENT:} b \neq a \Rightarrow \neg Hit(b, Ball)$$
$$\text{PRECOND:} Approaching(Ball, loc) \wedge At(a, loc)$$
$$\text{EFFECT:} Returned(Ball)) .$$

$$Action(Carry(a, cooler, here, there),$$
$$\text{CONCURRENT:} b \neq a \wedge Carry(b, cooler, here, there)$$
$$\text{PRECOND:} At(a, here) \wedge At(cooler, here) \wedge Cooler(cooler)$$
$$\text{EFFECT:} At(a, there) \wedge At(cooler, there) \wedge \neg At(a, here) \wedge \neg At(cooler, here)).$$

  - Hit(a, Ball) has an exclusion: no other player can Hit simultaneously.
  - Carry(a, cooler) requires another actor to perform the same action concurrently.

- With such schemas, standard planning algorithms can handle multiactor plans with minimal changes.
- When actors are only loosely coupled, single-agent heuristics remain effective, though advanced refinements (like HTNs or replanning) are beyond this discussion.

4.6.2 Planning with multiple agents: Cooperation and coordination

- In a **true multiagent setting**, each agent plans independently, even if they share the same goals and knowledge.
- At first, this might seem like the multibody case—agents could just compute a single joint plan and execute their assigned parts.
- However, the phrase "the joint solution" is misleading because multiple joint solutions can exist.
- For example, in a tennis doubles team, there are several valid ways to achieve the goal (e.g., either player can return the ball), so no unique joint plan naturally emerges.

$$\text{PLAN 2:}$$
$$A : [Go(A, LeftNet), NoOp(A)]$$
$$B : [Go(B, RightBaseline), Hit(B, Ball)] .$$

- If both agents agree on the same plan, the goal is achieved. But if they choose different plans, failure or conflict occurs—like both tennis players trying to hit the ball or neither doing so.
- To coordinate, agents can use several mechanisms:

1. Conventions:

- Pre-established rules that restrict plan choices (e.g., "stay on your side of the court" or "drive on the left/right").
- When widely adopted, such conventions become social laws.

2. Communication:

- Agents can communicate to reach a shared plan—verbally (e.g., “Mine!” or “Yours!”) or nonverbally.
- Observing another’s action and inferring their plan is called plan recognition.
- For example, if one player moves to the net, the other infers they should cover the baseline.

3. Evolutionary conventions:

- Cooperation can emerge naturally over time, as seen in ant colonies, where local interactions and learned priorities lead to efficient collective behavior.

4. Flocking (boids):

- In bird flocks, each agent (boid) adjusts its movement based on nearby neighbors, combining three simple rules: alignment, cohesion, and separation, leading to coordinated group motion without central control.



Figure 11.11 (a) A simulated flock of birds, using Reynold’s boids model. Image courtesy Giuseppe Randazzo, novastructura.net. (b) An actual flock of starlings. Image by Eduardo (pastaboy sleeps on flickr). (c) Two competitive teams of agents attempting to capture the towers in the NERO game. Image courtesy Risto Miikkulainen.

1.Cohesion: a positive score for getting closer to the average position of the neighbors

2. Separation: a negative score for getting too close to any one neighbor

3. Alignment: a positive score for getting closer to the average heading of the neighbors

- When all boids follow the same simple local rules, the flock shows emergent behavior—moving like a single, cohesive body that maintains density and sometimes makes sudden collective turns.
- No agent needs a shared or explicit joint plan.
- The hardest multiagent problems combine cooperation (within a team) and competition (against opponents) without any central control—like in robotic soccer or the NERO game. Efficient planning methods for such mixed cooperative–competitive environments are still at an early stage of development.